Chapter 17. Working with Large Databases

In the early days of relational databases, hard drive capacity was measured in megabytes, and databases were generally easy to administer simply because they couldn't get very large. Today, however, hard drive capacity has ballooned to 15 TB, a modern disk array can store more than 4 PB of data, and storage in the cloud is essentially limitless. While relational databases face various challenges as data volumes continue to grow, there are strategies such as partitioning, clustering, and sharding that allow companies to continue to utilize relational databases by spreading data across multiple storage tiers and servers. Other companies have decided to move to big data platforms such as Hadoop in order to handle huge data volumes. This chapter looks at some of these strategies, with an emphasis on techniques for scaling relational databases.

Partitioning

When exactly does a database table become "too big"? If you ask this question to 10 different data architects/administrators/developers, you will likely get 10 different answers. Most people, however, would agree that the following tasks become more difficult and/or time consuming as a table grows past a few million rows:

- Query execution requiring full table scans
- Index creation/rebuild
- Data archival/deletion
- Generation of table/index statistics
- Table relocation (e.g., move to a different tablespace)
- Database backups

These tasks can start as routine when a database is small, then become time consuming as more data accumulates, and then become problematic/impossible due to limited administrative time windows. The best way to prevent administrative issues from occurring in the future is to break large tables into pieces, or *partitions*, when the table is first created (although tables can be partitioned later, it is easier to do so initially). Administrative tasks can be performed on individual partitions, often in parallel, and some tasks can skip one or more partitions entirely.

Partitioning Concepts

Table partitioning was introduced in the late 1990s by Oracle, but since then every major database server has added the ability to partition tables and indexes. When a table is partitioned, two or more table partitions are created, each having the exact same definition but with nonoverlapping subsets of data. For example, a table containing sales data could be partitioned by month using the column containing the sale date, or it could be partitioned by geographic region using the state/province code.

Once a table has been partitioned, the table itself becomes a virtual concept; the partitions hold the data, and any indexes are built on the data in the partitions. However, the database users can still interact with the table without knowing that the table had been partitioned. This is similar in concept to a view, in that the users interact with schema objects that are interfaces rather than actual tables. While every partition must have the same schema definition (columns, column types, etc.), there are several administrative features that can differ for each partition:

- Partitions may be stored on different tablespaces, which can be on different physical storage tiers.
- Partitions can be compressed using different compression schemes.
- Local indexes (more on this shortly) can be dropped for some partitions.
- Table statistics can be frozen on some partitions, while being periodically refreshed on others.
- Individual partitions can be pinned into memory or stored in the database's flash storage tier.

Thus, table partitioning allows for flexibility with data storage and administration, while still presenting the simplicity of a single table to your user community.

Table Partitioning

The partitioning scheme available in most relational databases is *horizontal partitioning*, which assigns entire rows to exactly one partition. Tables may also be partitioned *vertically*, which involves assigning sets of columns to different partitions, but this must be done manually. When partitioning a table horizontally, you must choose a *partition key*, which is the column whose values are used to assign a row to a particular partition. In most cases, a table's partition key consists of a single column, and a *partitioning function* is applied to this column to determine in which partition each row should reside.

Index Partitioning

If your partitioned table has indexes, you will get to choose whether a particular index should stay intact, known as a *global index*, or be broken into pieces such that each partition has its own index, which is called a *local index*. Global indexes span all partitions of the table and are useful for queries that do not specify a value for the partition key. For example, let's say your table is partitioned on the `sale_date` column, and a user executes the following query:

```
SELECT sum(amount) FROM sales WHERE geo_region_cd = 'US'
```

Since this query does not include a filter condition on the `sale_date` column, the server will need to search every partition in order to find the total US sales. If a global index is built on the `geo_region_cd` column, however, then the server could use this index to quickly find all of the rows containing US sales.

Partitioning Methods

While each database server has its own unique partitioning features, the next three sections describe the common partitioning methods available across most servers.

Range partitioning

Range partitioning was the first partitioning method to be implemented, and it is still one of the most widely used. While range partitioning can be used for several different column types, the most common usage is to break up tables by date ranges. For example, a table named `sales` could be partitioned using the `sale_date` column such that data for each week is stored in a different partition:

```
mysql> CREATE TABLE sales
    ->  (sale_id INT NOT NULL,
    ->   cust_id INT NOT NULL,
    ->   store_id INT NOT NULL,
    ->   sale_date DATE NOT NULL,
    ->   amount DECIMAL(9,2)
    ->  )
    -> PARTITION BY RANGE (yearweek(sale_date))
    ->  (PARTITION s1 VALUES LESS THAN (202002),
    ->   PARTITION s2 VALUES LESS THAN (202003),
    ->   PARTITION s3 VALUES LESS THAN (202004),
```

```
    ->   PARTITION s4 VALUES LESS THAN (202005),
    ->   PARTITION s5 VALUES LESS THAN (202006),
    ->   PARTITION s999 VALUES LESS THAN (MAXVALUE)
    ->  );
Query OK, 0 rows affected (1.78 sec)
```

This statement creates six different partitions, one for each of the first five weeks of 2020 and a sixth partition named s999 to hold any rows beyond week five of year 2020. For this table, the `yearweek(sale_date)` expression is used as the partitioning function, and the `sale_date` column serves as the partitioning key. To see the metadata about your partitioned tables, you can use the `partitions` table in the `information_schema` database:

```
mysql> SELECT partition_name, partition_method, partition_expression
    -> FROM information_schema.partitions
    -> WHERE table_name = 'sales'
    -> ORDER BY partition_ordinal_position;
+----------------+-----------------+-----------------------+
| PARTITION_NAME | PARTITION_METHOD | PARTITION_EXPRESSION    |
+----------------+-----------------+-----------------------+
| s1             | RANGE           | yearweek(`sale_date`,0) |
| s2             | RANGE           | yearweek(`sale_date`,0) |
| s3             | RANGE           | yearweek(`sale_date`,0) |
| s4             | RANGE           | yearweek(`sale_date`,0) |
| s5             | RANGE           | yearweek(`sale_date`,0) |
| s999           | RANGE           | yearweek(`sale_date`,0) |
+----------------+-----------------+-----------------------+
6 rows in set (0.00 sec)
```

One of the administrative tasks that will need to be performed on the `sales` table involves generating new partitions to hold future data (to keep data from being added to the `maxvalue` partition). Different databases handle this in different ways, but in MySQL you could use the `reorganize partition` clause of the `alter table` command to split the s999 partition into three pieces:

```
ALTER TABLE sales REORGANIZE PARTITION s999 INTO
 (PARTITION s6 VALUES LESS THAN (202007),
  PARTITION s7 VALUES LESS THAN (202008),
  PARTITION s999 VALUES LESS THAN (MAXVALUE)
 );
```

If you execute the previous metadata query again, you will now see eight partitions:

```
mysql> SELECT partition_name, partition_method, partition_expression
    -> FROM information_schema.partitions
    -> WHERE table_name = 'sales'
    -> ORDER BY partition_ordinal_position;
+----------------+-----------------+-----------------------+
| PARTITION_NAME | PARTITION_METHOD | PARTITION_EXPRESSION    |
+----------------+-----------------+-----------------------+
| s1             | RANGE           | yearweek(`sale_date`,0) |
| s2             | RANGE           | yearweek(`sale_date`,0) |
| s3             | RANGE           | yearweek(`sale_date`,0) |
| s4             | RANGE           | yearweek(`sale_date`,0) |
| s5             | RANGE           | yearweek(`sale_date`,0) |
| s6             | RANGE           | yearweek(`sale_date`,0) |
| s7             | RANGE           | yearweek(`sale_date`,0) |
| s999           | RANGE           | yearweek(`sale_date`,0) |
+----------------+-----------------+-----------------------+
8 rows in set (0.00 sec)
```

Next, let's add a couple of rows to the table:

```
mysql> INSERT INTO sales
    -> VALUES
    ->  (1, 1, 1, '2020-01-18', 2765.15),
    ->  (2, 3, 4, '2020-02-07', 5322.08);
Query OK, 2 rows affected (0.18 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

The table now has two rows, but into which partitions were they inserted? To find out, let's use the `partition` subclause of the `from` clause to count the number of rows in each partition:

```
mysql> SELECT concat('# of rows in S1 = ', count(*)) partition_rowcount
    -> FROM sales PARTITION (s1) UNION ALL
    -> SELECT concat('# of rows in S2 = ', count(*)) partition_rowcount
    -> FROM sales PARTITION (s2) UNION ALL
    -> SELECT concat('# of rows in S3 = ', count(*)) partition_rowcount
    -> FROM sales PARTITION (s3) UNION ALL
    -> SELECT concat('# of rows in S4 = ', count(*)) partition_rowcount
    -> FROM sales PARTITION (s4) UNION ALL
    -> SELECT concat('# of rows in S5 = ', count(*)) partition_rowcount
```

```
    -> FROM sales PARTITION (s5) UNION ALL
    -> SELECT concat('# of rows in S6 = ', count(*)) partition_rowcount
    -> FROM sales PARTITION (s6) UNION ALL
    -> SELECT concat('# of rows in S7 = ', count(*)) partition_rowcount
    -> FROM sales PARTITION (s7) UNION ALL
    -> SELECT concat('# of rows in S999 = ', count(*)) partition_rowcount
    -> FROM sales PARTITION (s999);
+----------------------+
| partition_rowcount   |
+----------------------+
| # of rows in S1 = 0  |
| # of rows in S2 = 1  |
| # of rows in S3 = 0  |
| # of rows in S4 = 0  |
| # of rows in S5 = 1  |
| # of rows in S6 = 0  |
| # of rows in S7 = 0  |
| # of rows in S999 = 0 |
+----------------------+
8 rows in set (0.00 sec)
```

The results show that one row was inserted into partition S2, and the other row was inserted into the S5 partition. The ability to query a specific partition involves knowing the partitioning scheme, so it is unlikely that your user community will be executing these types of queries, but they are commonly used for administrative types of activities.

List partitioning

If the column chosen as the partitioning key contains state codes (e.g., CA, TX, VA, etc.), currencies (e.g., USD, EUR, JPY, etc.), or some other enumerated set of values, you may want to utilize list partitioning, which allows you to specify which values will be assigned to each partition. For example, let's say that the `sales` table includes the column `geo_region_cd`, which contains the following values:

```
+---------------+-------------------------+
| geo_region_cd | description             |
+---------------+-------------------------+
| US_NE         | United States North East |
| US_SE         | United States South East |
| US_MW         | United States Mid West   |
| US_NW         | United States North West |
| US_SW         | United States South West |
| CAN           | Canada                   |
```

```
| MEX          | Mexico                 |
| EUR_E        | Eastern Europe         |
| EUR_W        | Western Europe         |
| CHN          | China                  |
| JPN          | Japan                  |
| IND          | India                  |
| KOR          | Korea                  |
+--------------+------------------------+
13 rows in set (0.00 sec)
```

You could group these values into geographic regions and create a partition for each one, as in:

```
mysql> CREATE TABLE sales
    ->  (sale_id INT NOT NULL,
    ->   cust_id INT NOT NULL,
    ->   store_id INT NOT NULL,
    ->   sale_date DATE NOT NULL,
    ->   geo_region_cd VARCHAR(6) NOT NULL,
    ->   amount DECIMAL(9,2)
    ->  )
    -> PARTITION BY LIST COLUMNS (geo_region_cd)
    ->  (PARTITION NORTHAMERICA VALUES IN ('US_NE','US_SE','US_MW',
    ->                             'US_NW','US_SW','CAN','MEX'),
    ->   PARTITION EUROPE VALUES IN ('EUR_E','EUR_W'),
    ->   PARTITION ASIA VALUES IN ('CHN','JPN','IND')
    ->  );
Query OK, 0 rows affected (1.13 sec)
```

The table has three partitions, where each partition includes a set of two or more `geo_region_cd` values. Next, let's add a few rows to the table:

```
mysql> INSERT INTO sales
    -> VALUES
    ->  (1, 1, 1, '2020-01-18', 'US_NE', 2765.15),
    ->  (2, 3, 4, '2020-02-07', 'CAN', 5322.08),
    ->  (3, 6, 27, '2020-03-11', 'KOR', 4267.12);
ERROR 1526 (HY000): Table has no partition for value from column_list
```

It looks like there was a problem, and the error message indicates that one of the geographic region codes was not assigned to a partition. Looking at the `create table` state-

ment, I see that I forgot to add Korea to the `asia` partition. This can be fixed using an `alter table` statement:

```
mysql> ALTER TABLE sales REORGANIZE PARTITION ASIA INTO
    ->  (PARTITION ASIA VALUES IN ('CHN','JPN','IND', 'KOR'));
Query OK, 0 rows affected (1.28 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

That seemed to do the trick, but let's check the metadata just to be sure:

```
mysql> SELECT partition_name, partition_expression,
    ->   partition_description
    -> FROM information_schema.partitions
    -> WHERE table_name = 'sales'
    -> ORDER BY partition_ordinal_position;
+----------------+---------------------+-------------------------------+
| PARTITION_NAME | PARTITION_EXPRESSION | PARTITION_DESCRIPTION         |
+----------------+---------------------+-------------------------------+
| NORTHAMERICA   | `geo_region_cd`     | 'US_NE','US_SE','US_MW','US_NW',|
|                |                     | 'US_SW','CAN','MEX'           |
| EUROPE         | `geo_region_cd`     | 'EUR_E','EUR_W'               |
| ASIA           | `geo_region_cd`     | 'CHN','JPN','IND','KOR'       |
+----------------+---------------------+-------------------------------+
3 rows in set (0.00 sec)
```

Korea has indeed been added to the `asia` partition, and the data insertion will now proceed without any issues:

```
mysql> INSERT INTO sales
    -> VALUES
    ->  (1, 1, 1, '2020-01-18', 'US_NE', 2765.15),
    ->  (2, 3, 4, '2020-02-07', 'CAN', 5322.08),
    ->  (3, 6, 27, '2020-03-11', 'KOR', 4267.12);
Query OK, 3 rows affected (0.26 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

While range partitioning allows for a `maxvalue` partition to catch any rows that don't map to any other partition, it's important to keep in mind that list partitioning doesn't provide for a spillover partition. Thus, any time you need to add another column value (e.g., the company starts selling products in Australia), you will need to modify the partitioning definition before rows with the new value can be added to the table.

Hash partitioning

If your partition key column doesn't lend itself to range or list partitioning, there is a third option that endeavors to distribute rows evenly across a set of partitions. The server does this by applying a *hashing function* to the column value, and this type of partitioning is (not surprisingly) called *hash partitioning*. Unlike list partitioning, where the column chosen as the partitioning key should contain only a small number of values, hash partitioning works best when the partitioning key column contains a large number of distinct values. Here's another version of the `sales` table but with four hash partitions generated by hashing the values in the `cust_id` column:

```
mysql> CREATE TABLE sales
    ->  (sale_id INT NOT NULL,
    ->   cust_id INT NOT NULL,
    ->   store_id INT NOT NULL,
    ->   sale_date DATE NOT NULL,
    ->   amount DECIMAL(9,2)
    ->  )
    -> PARTITION BY HASH (cust_id)
    ->   PARTITIONS 4
    ->    (PARTITION H1,
    ->     PARTITION H2,
    ->     PARTITION H3,
    ->     PARTITION H4
    ->    );
Query OK, 0 rows affected (1.50 sec)
```

When rows are added to the `sales` table, they will be evenly distributed across the four partitions, which I named `H1`, `H2`, `H3`, and `H4`. In order to see how good a job it does, let's add 16 rows, each with a different value for the `cust_id` column:

```
mysql> INSERT INTO sales
    -> VALUES
    ->  (1, 1, 1, '2020-01-18', 1.1), (2, 3, 4, '2020-02-07', 1.2),
    ->  (3, 17, 5, '2020-01-19', 1.3), (4, 23, 2, '2020-02-08', 1.4),
    ->  (5, 56, 1, '2020-01-20', 1.6), (6, 77, 5, '2020-02-09', 1.7),
    ->  (7, 122, 4, '2020-01-21', 1.8), (8, 153, 1, '2020-02-10', 1.9),
    ->  (9, 179, 5, '2020-01-22', 2.0), (10, 244, 2, '2020-02-11', 2.1),
    ->  (11, 263, 1, '2020-01-23', 2.2), (12, 312, 4, '2020-02-12', 2.3),
    ->  (13, 346, 2, '2020-01-24', 2.4), (14, 389, 3, '2020-02-13', 2.5),
    ->  (15, 472, 1, '2020-01-25', 2.6), (16, 502, 1, '2020-02-14', 2.7);
```

```
Query OK, 16 rows affected (0.19 sec)
Records: 16  Duplicates: 0  Warnings: 0
```

If the hashing function does a good job of distributing the rows evenly, we should ideally see four rows in each of the four partitions:

```
mysql> SELECT concat('# of rows in H1 = ', count(*)) partition_rowcount
    -> FROM sales PARTITION (h1) UNION ALL
    -> SELECT concat('# of rows in H2 = ', count(*)) partition_rowcount
    -> FROM sales PARTITION (h2) UNION ALL
    -> SELECT concat('# of rows in H3 = ', count(*)) partition_rowcount
    -> FROM sales PARTITION (h3) UNION ALL
    -> SELECT concat('# of rows in H4 = ', count(*)) partition_rowcount
    -> FROM sales PARTITION (h4);
+---------------------+
| partition_rowcount  |
+---------------------+
| # of rows in H1 = 4 |
| # of rows in H2 = 5 |
| # of rows in H3 = 3 |
| # of rows in H4 = 4 |
+---------------------+
4 rows in set (0.00 sec)
```

Given that only 16 rows were inserted, this is a pretty good distribution, and as the number of rows increases, each partition should contain close to 25% of the rows as long as there are a reasonably large number of distinct values for the `cust_id` column.

Composite partitioning

If you need finer-grained control of how data is allocated to your partitions, you can employ *composite partitioning*, which allows you to use two different types of partitioning for the same table. With composite partitioning, the first partitioning method defines the partitions, and the second partitioning method defines the *subpartitions*. Here's an example, again using the `sales` table, utilizing both range and hash partitioning:

```
mysql> CREATE TABLE sales
    ->  (sale_id INT NOT NULL,
    ->   cust_id INT NOT NULL,
    ->   store_id INT NOT NULL,
    ->   sale_date DATE NOT NULL,
```

```
    ->   amount DECIMAL(9,2)
    ->  )
    -> PARTITION BY RANGE (yearweek(sale_date))
    -> SUBPARTITION BY HASH (cust_id)
    ->  (PARTITION s1 VALUES LESS THAN (202002)
    ->    (SUBPARTITION s1_h1,
    ->     SUBPARTITION s1_h2,
    ->     SUBPARTITION s1_h3,
    ->     SUBPARTITION s1_h4),
    ->   PARTITION s2 VALUES LESS THAN (202003)
    ->    (SUBPARTITION s2_h1,
    ->     SUBPARTITION s2_h2,
    ->     SUBPARTITION s2_h3,
    ->     SUBPARTITION s2_h4),
    ->   PARTITION s3 VALUES LESS THAN (202004)
    ->    (SUBPARTITION s3_h1,
    ->     SUBPARTITION s3_h2,
    ->     SUBPARTITION s3_h3,
    ->     SUBPARTITION s3_h4),
    ->   PARTITION s4 VALUES LESS THAN (202005)
    ->    (SUBPARTITION s4_h1,
    ->     SUBPARTITION s4_h2,
    ->     SUBPARTITION s4_h3,
    ->     SUBPARTITION s4_h4),
    ->   PARTITION s5 VALUES LESS THAN (202006)
    ->    (SUBPARTITION s5_h1,
    ->     SUBPARTITION s5_h2,
    ->     SUBPARTITION s5_h3,
    ->     SUBPARTITION s5_h4),
    ->   PARTITION s999 VALUES LESS THAN (MAXVALUE)
    ->    (SUBPARTITION s999_h1,
    ->     SUBPARTITION s999_h2,
    ->     SUBPARTITION s999_h3,
    ->     SUBPARTITION s999_h4)
    ->  );
  Query OK, 0 rows affected (9.72 sec)
```

There are 6 partitions, each having 4 subpartitions, for a total of 24 subpartitions. Next, let's reinsert the 16 rows from the earlier example for hash partitioning:

```
  mysql> INSERT INTO sales
    -> VALUES
    ->  (1, 1, 1, '2020-01-18', 1.1), (2, 3, 4, '2020-02-07', 1.2),
    ->  (3, 17, 5, '2020-01-19', 1.3), (4, 23, 2, '2020-02-08', 1.4),
    ->  (5, 56, 1, '2020-01-20', 1.6), (6, 77, 5, '2020-02-09', 1.7),
```

```
    -> (7, 122, 4, '2020-01-21', 1.8), (8, 153, 1, '2020-02-10', 1.9),
    -> (9, 179, 5, '2020-01-22', 2.0), (10, 244, 2, '2020-02-11', 2.1),
    -> (11, 263, 1, '2020-01-23', 2.2), (12, 312, 4, '2020-02-12', 2.3),
    -> (13, 346, 2, '2020-01-24', 2.4), (14, 389, 3, '2020-02-13', 2.5),
    -> (15, 472, 1, '2020-01-25', 2.6), (16, 502, 1, '2020-02-14', 2.7);
Query OK, 16 rows affected (0.22 sec)
Records: 16  Duplicates: 0  Warnings: 0
```

When you query the `sales` table, you can retrieve data from one of the partitions, in which case you retrieve data from the four subpartitions associated with the partition:

```
mysql> SELECT *
    -> FROM sales PARTITION (s3);
+---------+---------+----------+------------+--------+
| sale_id | cust_id | store_id | sale_date  | amount |
+---------+---------+----------+------------+--------+
|       5 |      56 |        1 | 2020-01-20 |   1.60 |
|      15 |     472 |        1 | 2020-01-25 |   2.60 |
|       3 |      17 |        5 | 2020-01-19 |   1.30 |
|       7 |     122 |        4 | 2020-01-21 |   1.80 |
|      13 |     346 |        2 | 2020-01-24 |   2.40 |
|       9 |     179 |        5 | 2020-01-22 |   2.00 |
|      11 |     263 |        1 | 2020-01-23 |   2.20 |
+---------+---------+----------+------------+--------+
7 rows in set (0.00 sec)
```

Because the table is subpartitioned, you may also retrieve data from a single subpartition:

```
mysql> SELECT *
    -> FROM sales PARTITION (s3_h3);
+---------+---------+----------+------------+--------+
| sale_id | cust_id | store_id | sale_date  | amount |
+---------+---------+----------+------------+--------+
|       7 |     122 |        4 | 2020-01-21 |   1.80 |
|      13 |     346 |        2 | 2020-01-24 |   2.40 |
+---------+---------+----------+------------+--------+
2 rows in set (0.00 sec)
```

This query retrieves data only from the `s3_h3` subpartition of the `s3` partition.

Partitioning Benefits

One major advantage to partitioning is that you may only need to interact with as few as one partition, rather than the entire table. For example, if your table is range-partitioned on the `sales_date` column and you execute a query that includes a filter condition such as `WHERE sales_date BETWEEN '2019-12-01' AND '2020-01-15'`, the server will check the table's metadata to determine which partitions actually need to be included. This concept is called *partition pruning*, and it is one of the biggest advantages of table partitioning.

Similarly, if you execute a query that includes a join to a partitioned table and the query includes a condition on the partitioning column, the server can exclude any partitions that do not contain data pertinent to the query. This is known as *partition-wise joins*, and it is similar to partition pruning in that only those partitions that contain data needed by the query will be included.

From an administrative standpoint, one of the main benefits to partitioning is the ability to quickly delete data that is no longer needed. For example, financial data may need to be kept online for seven years; if a table has been partitioned based on transaction dates, any partitions holding data greater than seven years old can be dropped. Another administrative advantage to partitioned tables is the ability to perform updates on multiple partitions simultaneously, which can greatly reduce the time needed to touch every row in a table.

Clustering

With enough storage combined with a reasonable partitioning strategy, you can store a great deal of data in a single relational database. But what happens if you need to handle thousands of concurrent users or generate tens of thousands of reports during a nightly cycle? Even if you have sufficient data storage, you may not have enough CPU, memory, or network bandwidth within a single server. One potential answer is *clustering*, which allows multiple servers to act as a single database.

Although there are several different clustering architectures, for the purposes of this discussion I am referring to shared-disk/shared-cache configurations, where every server in the cluster has access to all disks, and data cached in one server can be accessed by any other server in the cluster. With this type of architecture, an application server could attach to any one of the database servers in the cluster, with connections automatically failing over to another server in the cluster in case of failure. With an eight-server clus-

ter, you should be able to handle a very large number of concurrent users and associated queries/reports/jobs.

Of the commercial database vendors, Oracle is the leader in this space, with many of the world's biggest companies using the Oracle Exadata platform to host extremely large databases accessed by thousands of concurrent users. However, even this platform fails to meet the needs of the biggest companies, which led Google, Facebook, Amazon, and other companies to blaze new trails.

Sharding

Let's say you have been hired as the data architect for a new social media company. You are told to expect approximately one billion users, each of whom will generate 3.7 messages per day on average, and the data must be available indefinitely. After performing a few calculations, you determine that you would exhaust the biggest available relational database platform in less than a year. One possibility would be to partition not just individual tables but the entire database. Known as *sharding*, this approach partitions the data across multiple databases (called *shards*), so it is similar to table partitioning but on a larger scale and with far more complexity. If you were to employ this strategy for the social media company, you might decide to implement 100 separate databases, each one hosting the data for approximately 10 million users.

Sharding is a complex topic, and since this is an introductory book, I will refrain from going into detail, but here are a few of the issues that would need to be addressed:

- You will need to choose a *sharding key*, which is the value used to determine to which database to connect.
- While large tables will be divided into pieces, with individual rows assigned to a single shard, smaller reference tables may need to be replicated to all shards, and a strategy needs to be defined for how reference data can be modified and changes propagated to all shards.
- If individual shards become too large (e.g., the social media company now has two billion users), you will need a plan for adding more shards and redistributing data across the shards.
- When you need to make schema changes, you will need to have a strategy for deploying the changes across all of the shards so that all schemas stay in sync.
- If application logic needs to access data stored in two or more shards, you need to have a strategy for how to query across multiple databases and also how to implement transactions across multiple databases.

If this seems complicated, that's because it is, and by the late 2000s many companies began looking for new approaches. The next section looks at other strategies for handling very large data sets completely outside the realm of relational databases.

Big Data

After spending some time weighing the pros and cons of sharding, let's say that you (the data architect of the social media company) decide to investigate other approaches. Rather than attempting to forge your own path, you might benefit from reviewing the work done by other companies that deal with massive amounts of data: companies like Amazon, Google, Facebook, and Twitter. Together, the set of technologies pioneered by these companies (and others) has been branded as *big data*, which has become an industry buzzword but has several possible definitions. One way to define the boundaries of big data is with the "3 Vs":

Volume

In this context, volume generally means billions or trillions of data points.

Velocity

This is a measure of how quickly data arrives.

Variety

This means that data is not always structured (as in rows and columns in a relational database) but can also be unstructured (e.g., emails, videos, photos, audio files, etc.).

So, one way to characterize big data is any system designed to handle a huge amount of data of various formats arriving at a rapid pace. The following sections offer a quick description of some of the big data technologies that have evolved over the past 15 years or so.

Hadoop

Hadoop is best described as an *ecosystem*, or a set of technologies and tools that work together. Some of the major components of Hadoop include:

Hadoop Distributed File System (HDFS)

Like the name implies, HDFS enables file management across a large number of servers.

MapReduce

This technology processes large amounts of structured and unstructured data by breaking a task into many small pieces that can be run in parallel across many servers.

YARN

This is a resource manager and job scheduler for HDFS.

Together, these technologies allow for the storage and processing of files across hundreds or even thousands of servers acting as a single logical system. While Hadoop is widely used, querying the data using MapReduce generally requires a programmer, which has led to the development of several SQL interfaces, including Hive, Impala, and Drill.

NoSQL and Document Databases

In a relational database, data must generally conform to a predefined schema consisting of tables made up of columns holding numbers, strings, dates, etc. What happens, however, if the structure of the data isn't known beforehand or if the structure is known but changes frequently? The answer for many companies is to combine both the data and schema definition into documents using a format such as XML or JSON and then store the documents in a database. By doing so, various types of data can be stored in the same database without the need to make schema modifications, which makes storage easier but puts the burden on query and analytic tools to make sense of the data stored in the documents.

Document databases are a subset of what are called NoSQL databases, which typically store data using a simple key-value mechanism. For example, using a document database such as MongoDB, you could utilize the customer ID as the key to store a JSON document containing all of the customer's data, and other users can read the schema stored within the document to make sense of the data stored within.

Cloud Computing

Prior to the advent of big data, most companies had to build their own data centers to house the database, web, and application servers used across the enterprise. With the advent of cloud computing, you can choose to essentially outsource your data center to platforms such as Amazon Web Services (AWS), Microsoft Azure, or Google Cloud. One of the biggest benefits to hosting your services in the cloud is instant scalability, which allows you to quickly dial up or down the amount of computing power needed to run your services. Startups love these platforms because they can start writing code without spending any money up front for servers, storage, networks, or software licenses.

As far as databases are concerned, a quick look at AWS's database and analytics offerings yields the following options:

- Relational databases (MySQL, Aurora, PostgreSQL, MariaDB, Oracle, and SQL Server)
- In-memory database (ElastiCache)
- Data warehousing database (Redshift)
- NoSQL database (DynamoDB)
- Document database (DocumentDB)
- Graph database (Neptune)
- Time-series database (TimeStream)
- Hadoop (EMR)
- Data lakes (Lake Formation)

While relational databases dominated the landscape up until the mid-2000s, it's pretty easy to see that companies are now mixing and matching various platforms and that relational databases may become less popular over time.

Conclusion

Databases are getting larger, but at the same time storage, clustering, and partitioning technologies are becoming more robust. Working with huge amounts of data can be quite challenging, regardless of the technology stack. Whether you use relational databases, big data platforms, or a variety of database servers, SQL is evolving to facilitate data retrieval from various technologies. This will be the subject of the last chapter in this book, where I will demonstrate the use of a SQL engine to query data stored in multiple formats.

Previous chapter

< 16. Analytic Functions

Next chapter

18. SQL and Big Data >